Spova Clone

Full platform plan: marketing, client portal, live trader, admin

This PDF is a printable copy of **docs/PLATFORM_PLAN.md**. That markdown file is the source of truth for you and for the Cursor agent (rule: **.cursor/rules/spova-platform.mdc**). Last updated 5 October 2026. Status: direction locked, repo not yet scaffolded.

Honest scope: this is a broker platform, not a static website. Spova itself is WordPress + Fintech360 CRM + MatchTrader + a third-party cashier. We clone the product shape with our own stack. We do not buy those vendors unless you pay for the licences.

1. Four products

#	Product	What the user sees
1	**Marketing site**	Public pages like spova.com: Home, Markets, Accounts, Platform, Legal, FAQ
2	**Client portal**	Login, KYC/docs, demo/live accounts, deposit, withdraw, history, statements, help
3	**Trading terminal**	Live charts, buy/sell, positions, margin, P/L
4	**Admin back office**	Approve KYC, inspect each client, credit/debit, freeze trading, change leverage/margins

We build all four. We do **not** build them in parallel from day one.

2. What Spova actually is (research 5 Oct 2026)

Visited https://spova.com and signed in to the client area as deactivate@gmail.com (Rankaj Kumar). No admin dashboard access.

Spova is not a custom one-off: WordPress marketing (multilingual), Fintech360 SPA at /client-area, MatchTrader at platform.spova.com, cashier at deposit.cashier.fintech360.tech (Finosoft cards + Praxis APM). Admin is vendor back office (Fintech360 + Match-Trader Manager).

Client-area rail: Deposit, Trading accounts, Payments, Documents, Statement, InvestiQ, Help, Profile.

This test account: live Classic MatchTrade, leverage 1:100, balance $0; demo login 746181 ~$16.15M (simulated, no withdraw); KYC POI pending / POR incomplete; deposits blocked by cashier BO; many declined $250 deposits in May 2026. Marketing: FX leverage up to 1:400, margin call 100%, stop-out 20%.

3. Architecture (locked)

One API. One Postgres. Four UIs (or one Next.js app with four route groups). Same users, same money. Day-1 trading: CFD on our books. We stream prices; the client opens a position; we are the counterparty. Admin controls balance. No licensed FX matching engine in v1.

```
Cloudflare CDN
   +-- marketing  /
   +-- client     /app
   +-- trader     /trade
   +-- admin      /admin
            |
        Nginx / Caddy
            |
        NestJS API  (REST + WebSocket)
            +-- PostgreSQL 16
            +-- Redis 7
            +-- BullMQ workers (stop-out, mail, PDF)
            +-- R2/S3  (KYC files)
            +-- Price worker (Binance / Finnhub / Twelve Data)
```

4. Stack (do not bikeshed)

Layer	Choice
Repo	pnpm + Turborepo
All UIs	Next.js 15 App Router + Tailwind + shadcn
Charts	TradingView Lightweight Charts
API	NestJS + Prisma
Database	PostgreSQL 16; money as NUMERIC(18,2)
Cache / pubsub	Redis 7
Jobs	BullMQ
Files	Cloudflare R2 (or S3) for KYC — never git/disk
Auth	JWT httpOnly cookies; access ~15m, refresh ~7d
Realtime	Nest WS / Socket.IO rooms user:{id}, admin, symbol:EURUSD
Payments v1	Admin credit/debit + deposit/withdraw request queue
Payments v2	One PSP adapter later (Stripe / crypto / UPI)
Email	Resend or SMTP
Infra v1	One 8GB VPS + Docker Compose + Cloudflare
Backups	Nightly pg_dump to R2

Forbidden for the ledger: MongoDB, Firebase as source of truth, UPDATE balance without a ledger row, KYC stored in the repo.

5. Target routes

```
/                       marketing
/forex  /commodities ...  market pages
/login  /register         auth
/app                      client CRM (Spova-style left rail)
   /app/accounts
   /app/payments/deposit | withdraw | history
   /app/documents
   /app/statement
   /app/help
```

```
   /app/profile
/trade                    trader workspace
/admin                    role-guarded
   /admin/clients  /kyc  /wallet  /trades  /payments  /settings
```

6. NestJS modules

1. Auth — register, login, refresh, forgot password, roles
2. Users / Profile
3. KYC — POI / POR / payment / other
4. Accounts — demo + live, leverage, status, trading_blocked
5. Ledger — the only place money moves
6. Payments — deposit/withdraw requests; gateway adapters later
7. Market — symbols, contract specs, sessions
8. Trading — market/limit, positions, SL/TP
9. Risk — used/free margin, call 100%, stop-out 20% (per-tier config)
10. Admin — freeze, credit/debit with reason, force-close, change leverage
11. Support — tickets
12. Statements — PDF
13. Realtime — ticks + position updates

7. ERD (core)

USER 1—* TRADING_ACCOUNT, KYC_DOCUMENT, TICKET, SESSION; USER 1—1 PROFILE. TRADING_ACCOUNT 1—* LEDGER_ENTRY, ORDER, POSITION, PAYMENT_REQUEST. SYMBOL 1—* ORDER, POSITION, CANDLE. ORDER 0..1—POSITION. ADMIN_AUDIT *—USER.

Main tables

Table	Key fields
USER	id uuid, email unique, password_hash, role client|admin|superadmin, status active|frozen|closed, kyc_poi, kyc_por
USER_PROFILE	user_id, first_name, last_name, phone, country, city, address, zip, dob
TRADING_ACCOUNT	type demo|live, login_code, tier classic..vip, currency, leverage, balance, credit, status, trading_blocked. Equity is computed: balance + floating_pnl + credit.
LEDGER_ENTRY	type deposit|withdraw|credit|debit|pnl|swap|fee|adjustment, amount, balance_after, reason, admin_id, ref_id
PAYMENT_REQUEST	kind deposit|withdraw, amount, status pending|approved|declined, method, admin_note
KYC_DOCUMENT	group poi|por|payment|other, doc_type, file_key, status pending|approved|rejected, reject_reason
SYMBOL	id EURUSD, asset_class, digits, contract_size, min_lot, max_lot, leverage_max, tick_size, is_open
ORDER / POSITION	side buy|sell, lots, prices, sl, tp, status, margin_used, profit

Also: password_reset, notification_prefs, admin_audit, statement_export, symbol_session.

Candles: Redis last price; optional candles table. Do not store every tick forever in v1.

Margin formulas (Spova-like, admin-configurable)

```
equity       = balance + floating_pnl + credit
used_margin  = sum(lots * contract_size * price / leverage)
free_margin  = equity - used_margin
margin_level = equity / used_margin * 100
default call = 100%     default stop-out = 20%
```

8. Build order (do not skip)

Phase	What	Why this order
0 ~1 week	Monorepo, Nest health, Postgres, Redis, Docker, login/register, empty /app /admin /trade, homepage	Runnable skeleton
1 first real milestone	Demo/live accounts, ledger, admin list clients, credit/debit, freeze, disable trading, change leverage	Without this, everything else is fake balances
2	KYC upload to R2, admin approve/reject, deposit/withdraw as requests + history	Matches how Spova BO actually works
3	Symbols, WS prices, candles, market buy/sell, positions, SL/TP, live P/L, stop-out. Demo first.	Trader is useless until money + risk exist
4	Marketing: forex, commodities, tiers, legal, FAQ	Easy; do after portal or you pixel-push for weeks
5	Live accounts + one payment adapter. InvestiQ extras last.	PSP before ledger is wasted work

9. Infra v1

Single VPS ~8 GB. Docker Compose: api, worker, web, postgres, redis. Cloudflare proxy. R2 bucket kyc. Nightly backup. Split machines only when WebSocket/charts actually hurt. Kubernetes on day 1 is ego, not a plan.

10. Time (honest)

One strong full-stack: **3–5 months** for a convincing demo of all four. Real-money production (audit, risk, 99.9% WS, licensed PSP): **6–12+ months**. Pixel-perfect Spova plus real FX liquidity is a broker company, not a website job.

11. Rules for implementation

1. Read docs/PLATFORM_PLAN.md before inventing a new stack or table.
2. Never move money except via LEDGER_ENTRY in a DB transaction with the account row locked.
3. Demo and live share code; they differ by trading_account.type and whether withdrawals are allowed.
4. Admin actions write admin_audit.
5. Do not integrate Praxis / Finosoft / MatchTrader unless the client buys them.
6. Do not start Kubernetes, a Go matching engine, or AI InvestiQ before Phase 1 works.
7. Do not commit .env, JWTs, or KYC files.
8. First scaffold: Turborepo + Nest + Prisma from this ERD + Next layouts + login + admin credit on demo. Stop until that is demoable.

When the client says start: scaffold Phase 0–1 only (Prisma models, auth + roles, admin credit/debit, client dashboard showing demo balance).